
Informe de Seguridad — SIGAI-SES

v1.0.0

Securitas Colombia S.A.



Informe de Seguridad — SIGAI-SES v1.0.0

1. Resumen Ejecutivo

[!IMPORTANT] El Sistema Integral de Gestión de Activos e Inventario (**SIGAI-SES**) implementa medidas de seguridad en **múltiples capas** para proteger la confidencialidad, integridad y disponibilidad de la información de **Securitas Colombia**.

Indicador	Estado
Controles de acceso	Implementados — OAuth2 + JWT + RBAC
Comunicación segura	HTTPS obligatorio en producción
Datos en reposo	bcrypt + hash SHA-256
Auditoría	Registro completo de acciones CRUD
OWASP Top 10	9/10 mitigados (1 no aplica)
Hallazgos corregidos	6/7 en auditoría Junio 2026

2. Controles de Acceso

2.1 Autenticación

Mecanismo	Implementación
Protocolo	OAuth2 Password Flow
Formato Token	JWT firmado con HS256
Access Token	Válido por 8 horas (configurable vía ACCESS_TOKEN_EXPIRE_MINUTES)
Refresh Token	Válido por 7 días , renovación sin credenciales
Almacenamiento	sessionStorage del navegador

Mecanismo	Implementación
Hash de Contraseñas	bcrypt con salt automático (passlib 1.7.4)
Sesiones	Tabla sesiones_usuario para revocación manual y auditoría
Rate Limiting	SlowAPI (10 req/min en endpoint de login)
Renovación	Interceptor Axios renueva tokens expirados automáticamente

2.2 Autorización (RBAC)

El sistema implementa **Control de Acceso Basado en Roles** con **3 niveles**:

Rol	Permisos	Nivel
ADMIN	Acceso total: CRUD usuarios, auditoría, configuración global, todos los módulos	Máximo
TÉCNICO	Consulta de inventario, registro de garantías, entregas, desmontes, reportes	Medio
TÉCNICO_LABORATORIO	Gestión de laboratorio, triaje de desmontes, consulta de inventario, garantías	Operativo

2.3 Aislamiento por Regional

[!NOTE] Los datos están protegidos por **código de regional**. Un usuario de “*Bogotá*” **no visualiza** equipos de “*Medellín*” a menos que tenga permisos de administrador nacional.

Implementación: Consultas SQL con filtro → WHERE id_regional = :regional_usuario

3. Seguridad en la Comunicación

3.1 Transporte

Aspecto	Estado	Badge
HTTPS	Obligatorio en producción (SSL/TLS con Let's Encrypt)	HTTPS Obligatorio
CORS	Restringido a orígenes configurados en CORS_ALLOWED_ORIGINS	CORS Configurado
HSTS	Planificado para v1.1.0	HSTS Planificado

3.2 API

Aspecto	Implementación	Estado
Validación de Entrada	Pydantic schemas con tipos estrictos	
Sanitización	SQLAlchemy ORM — query parameters	
Serialización	FastAPI serializa automáticamente, previniendo XSS	
Headers de Seguridad	Pendiente: CSP, X-Frame-Options, X-Content-Type-Options (v1.1.0)	
Rate Limiting	Login (SlowAPI, 10 req/min). Pendiente: general (v1.1.0)	
Bloqueo por intentos	Pendiente: bloqueo tras N intentos fallidos (v1.1.0)	

[!WARNING] Los aspectos marcados como **pendientes** representan riesgos aceptados temporalmente. Su implementación está planificada para la versión **v1.1.0**.

4. Seguridad de Datos

4.1 Datos en Reposo

Tipo de Dato	Protección	Estado
Contraseñas	Hash bcrypt (PBKDF2-based) con salt automático	Seguro
Tokens JWT	Almacenados hasheados (SHA-256) en <code>sesiones_usuario</code>	Seguro
Credenciales de Equipos	Pendiente de encriptación AES-256 en activos	Vulnerable
Datos Personales	Texto plano (identificación necesaria para operación)	Controlado
Logs de auditoría	Valores anterior/nuevo en formato JSON , inmutables	Seguro

4.2 Datos en Tránsito

- **Encriptación TLS/SSL** en producción
 - Tokens JWT enviados en header `Authorization: Bearer <token>`
 - Contraseñas solo se envían en endpoint de login, sobre **HTTPS**
 - No se transmiten contraseñas en texto plano después de la autenticación
-

5. Auditoría y Trazabilidad

5.1 Registro de Auditoría

[!TIP] Todas las operaciones **CRUD** y eventos de **login** quedan registrados en la tabla `audit_logs`.

Acción	Datos Registrados	Visible
CREATE	Usuario, tabla, timestamp, valor_nuevo	Siempre
UPDATE	Usuario, tabla, timestamp, valor_anterior, valor_nuevo	Siempre
DELETE	Usuario, tabla, timestamp, valor_anterior	Siempre
LOGIN	Usuario, tabla="usuarios", acción="LOGIN", timestamp	Siempre

5.2 Monitoreo Disponible

Endpoint	Propósito
GET /health	Verificación de estado del servidor
GET /health/db	Verificación de conexión a BD
GET /metrics	Métricas del sistema (CPU, RAM vía psutil)
Logs Uvicorn	app.log, error.log, access.log con rotación
Python JSON Logger	Logging estructurado

6. Evaluación OWASP Top 10

#	Categoría	Estado	Observaciones
A01	Broken Access Control	Mitigado	RBAC implementado, validación por regional, decorador <code>require_roles()</code>
A02	Cryptographic Failures	Mitigado parcialmente	bcrypt + HTTPS. Pendiente: encriptar credenciales técnicas
A03	Injection	Mitigado	SQLAlchemy ORM (parameter binding) + Pydantic + FastAPI sanitiza
A04	Insecure Design	Mitigado	Arquitectura por capas, validación en schemas, transacciones atómicas
A05	Security Misconfiguration	Mitigado parcialmente	CORS desde env, DEBUG=False. Pendiente: security headers HTTP
A06	Vulnerable Components	Mitigado	Dependencias con versiones fijas. Sin componentes vulnerables conocidos
A07	Identification Failures	Mitigado	JWT (8h), refresh tokens (7d), sesiones revocables, bcrypt
A08	Software Integrity Failures	Mitigado	CI/CD con GitHub Actions, code review via PRs, versiones fijas

#	Categoría	Estado	Observaciones
A09	Logging Failures	Mitigado	Auditoría completa CRUD + logs con rotación
A10	SSRF	No aplica	El sistema no realiza peticiones a URLs externas no controladas

[!NOTE] **Resumen OWASP:** 7 mitigados · 2 parciales · 1 no aplica

7. Hallazgos de Auditoría (14 Junio 2026)

7.1 Hallazgos Detectados y Corregidos

ID	Severidad	Descripción	Acción	Estado
H-01	Alta	Secretos en repositorio: .env con SECRET_KEY y AD-MIN_PASSWORD en 3 ubicaciones	Archivos saneados, creado .env.example	CORREGIDO
H-02	Alta	CORS permisivo: al-low_origins=['*'] en main.py	Ahora configurable desde CORS_ALLOWED_ORIGINS	CORREGIDO
H-03	Media	Credenciales de prueba en lo-custfile.py (admin123)	Ahora usa variables de entorno	CORREGIDO

ID	Severidad	Descripción	Acción	Estado
H-04	Baja	Debug prints: <code>console.log</code> en <code>Users.tsx</code>	Eliminados	CORREGIDO
H-05	Media	Lazy loading en relaciones async SQLAlchemy	Cambiado a <code>selectinload</code> en <code>crud_deliveries</code>	CORREGIDO

7.2 Vulnerabilidades Pendientes de Corrección (v1.1.0)

ID	Severidad	Descripción	Mitigación Propuesta
V-01	Media	Credenciales técnicas de activos en texto plano (<code>credenciales_tecnicas</code>)	Encriptar con AES-256 antes de almacenar
V-02	Media	Tokens JWT en <code>sessionStorage</code> (vulnerable a XSS)	Migrar a httpOnly cookies o implementar refresh token rotation
V-03	Baja	Rate limiting solo en login	Agregar middleware SlowAPI a toda la API
V-04	Baja	Security headers HTTP no configurados	Agregar middleware o configurar en Nginx
V-05	Baja	Sin bloqueo por intentos fallidos de login	Bloqueo temporal tras 5 intentos fallidos

[!NOTE] **V-06 (Pipeline CI/CD)** fue **CORREGIDO**: GitHub Actions con lint + tests + build ya está operativo (`.github/workflows/main.yml`).

7.3 Riesgos Aceptados

ID	Justificación
V-02	El equipo asume el riesgo de sessionStorage por simplicidad. Mitigado parcialmente por validación de tokens y DEBUG=False en producción
** Almacenamiento datos personales en texto plano**	Necesario para la operación del sistema. Mitigado por RBAC + auditoría completa

8. Protección de Datos Personales

8.1 Datos Personales Procesados

Tipo de Dato	Finalidad	Base Legal
Nombre, email, cédula, código empleado	Identificación de usuarios del sistema	Relación laboral / contractual
Datos de contacto de clientes	Gestión comercial y operativa	Relación contractual
Regional asignada	Control de acceso por ubicación	Necesidad operativa
Credenciales de equipos	Operación y mantenimiento	Necesidad del servicio
Registro de acciones (logs)	Auditoría y trazabilidad	Cumplimiento legal

8.2 Cumplimiento Legal (Colombia — Ley 1581 de 2012)

Requisito	Estado
Aviso de Privacidad	Documentado (ver 03_AVISO_DE_PRIVACIDAD.md)
Autorización del Titular	Gestionada por Securitas Colombia

Requisito	Estado
Derecho de Acceso	Gestión manual vía administrador
Derecho de Consulta	Solicitud vía soporte@ses-securitas.com.co
Derecho de Rectificación	Módulo de usuarios o vía administrador
Derecho de Actualización	Actualización directa o vía administrador
Derecho de Supresión	Soft delete en items, clientes, proveedores
Derecho de Revocación	Evaluación por oficial de protección de datos
Seguridad de Datos	Implementado parcialmente (ver sección 4)

9. Recomendaciones

Corto Plazo (v1.1.0)

[!WARNING] Estas vulnerabilidades deben ser atendidas **antes del próximo release**.

1. **Encriptar credenciales técnicas** — AES-256 para credenciales_tecnicas
2. **Security headers** — CSP, HSTS, X-Frame-Options, X-Content-Type-Options
3. **Rate limiting general** — SlowAPI a todos los endpoints
4. **Refresh token rotation** — Mitigar riesgo de sessionStorage
5. **Bloqueo por intentos fallidos** — Tras 5 intentos en login

Mediano Plazo (v1.2.0)

1. **Autenticación de dos factores (2FA)** — Implementar TOTP
2. **Monitoreo con Sentry** — Detección de errores en producción
3. **Pruebas de penetración** — Contratar firma externa
4. **Notificaciones push/email** — Integración SMTP para alertas
5. **Mejora de logging** — Implementar ELK stack o similar

Largo Plazo (v2.0.0)

1. **Certificación ISO 27001** — Iniciar proceso de certificación
 2. **Auditoría de seguridad anual** — Firma externa especializada
 3. **Módulo de gestión de consentimientos** — Datos personales
 4. **Cifrado de datos en reposo a nivel de BD** — TDE o similar
-

10. Conclusión

[!IMPORTANT] El sistema SIGAI-SES implementa **controles de seguridad sólidos** en autenticación (OAuth2 + JWT + bcrypt), autorización (RBAC + aislamiento regional) y auditoría (registro completo).

Indicador	Resultado
Auditoría técnica	14 Junio 2026
Hallazgos identificados	7
Hallazgos corregidos	6
Vulnerabilidades pendientes	5 (Media/Baja)
Riesgos aceptados	2
Vulnerabilidades críticas	0

**** Estado general:**** *Aceptable para producción con las mitigaciones documentadas.*

[!NOTE] **Documento actualizado al:** Julio 2026 · **v1.0** · Responsable: Oficial de Seguridad SIGAI-SES
